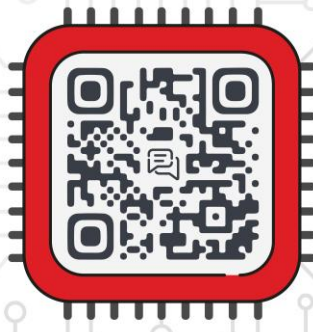




نظري

1

16

1920 sp

كلية الهندسة المعلوماتية

السنة الأولى

Introduction to Java

د. محمد الأحمد/د. عبد الرحمن
اللاحام



RB Informatcs; 09/04/2025

البرمجة 2

"Why Java?" لماذا جافا

- جافا هي لغة برمجة أنيقة وقوية:
- "Java is an elegant, powerful programming language"
- أبسط من لغات البرمجة غرضية التوجه الأخرى [على سبيل المثال، C++].
- "Java" هي الأساس للغات البرمجة الحديثة الأخرى [على سبيل المثال: C#].
- "Java" هي منصة مستقلة "اكتب مرة واحدة تعمل في كل مكان" "Write once run everywhere"
- تدعم "Java" منصات متعددة (Mac, Windows, Unix)، وأنواع متعددة من الأجهزة (أجهزة الحاسوب المكتبية، وأجهزة المساعد الرقمي الشخصي (PDA) والهواتف، والأجهزة المضمنة).
- تتمتع جافا بدعم جيد "Java has good support"
- وسائط متعددة وحزم رسومات جيدة.
- دعم جيد للتعامل والخادم والشبكة (applet, serverlet).
- بيئات التطوير المتكاملة (IDE) الجيدة والمجانية.

لغة البرمجة جافا: "المصممين": المصمّمون – Java Programming Language

Bill Joy

- من مؤسسي نظام BSD Unix في جامعة بيركلي.
- أسس شركة Sun Microsystems في أوائل الثمانينات.
- صاحب المقولة الشهيرة – "The network is the computer": كانت رؤية متقدمة عن عصرها.
- ركّزت Sun على سوق محطات العمل (Workstations).
- فاته قطار ثورة الحواسيب الشخصية (PC Revolution).
- بعد تراجع Sun ، انسحب إلى Aspen ، كولورادو.

James Gosling

- نال شهرة مبكرة كمؤلف Gosling Emacs (توقف بعد ظهور GNU Emacs)
- عمل لاحقاً على NeWS ، نظام نوافذ في Sun تم تجاوزه بواسطة X Window System

Java Programming Language: History

- انضم جوي وجوسلينغ إلى شركة Sun subsidiary, First Person, Inc. عام 1992.
- استهدف الإلكترونيات الاستهلاكية: أجهزة المساعد الرقمي الشخصي (PDA)، والأجهزة المنزلية، والهواتف، وجميعها مزودة بشبكات رخيصة تعمل بالأشعة تحت الحمراء.
- احتاج للغة صغيرة، وقوية، وآمنة، ومؤمنة، وسلكية.
- بدأ العمل على C++ وسرعان ما فقد الأمل، وقرر البدء من الصفر.
- الحظ السيء (مرة أخرى)
- سوء الحظ: لبرهة من الزمن ماتت أجهزة المساعد الرقمي الشخصي (PDA) مع زوال شركة Apple Newton تحول إلى التلفاز التفاعلي (ITV).
- سميت اللغة الناتجة ("Oak") ثم ماتت ITV أيضاً.
- الحظ السعيد (أخيراً)
- انفجرت الشبكة في عام 1993
- أصبح Oak جافا، وليحفظه التاريخ!

Machine Languages

- وحدة المعالجة المركزية (CPU) هي "عقل الكمبيوتر".
- لا يمكن لوحدة المعالجة المركزية فهم سوى التعليمات الأساسية جداً.
- على سبيل المثال: تخزين قيمة معينة في مواقع الذاكرة، إجراء بعض العمليات الحسابية، مقارنة قيمتين، البدء في تنفيذ التعليمات في موقع آخر.
- تشكل مجموعة تعليمات وحدة المعالجة المركزية لغة الآلة الخاصة بها.
- لغات الآلة أو التعليمات التي يفهمها الكمبيوتر، يتم تمثيلها بالأرقام.
- على سبيل المثال: بالنسبة لمعالجات Intel *86 (التي تتضمن Pentium) الأرقام
10110000 01100001
- تعطي الأرقام التعليمات التالية: نسخ الرقم 97 إلى وحدة تخزين سجل المعالج المسماة ah.
- كل نوع من وحدات المعالجة المركزية لديه لغة الآلة الخاصة به

فائدة أربيسيزية: كل نوع من المعالجات له لغة آلة خاصة به، لأن كل نوع من المعالجات يُصمم ببنية داخلية فريدة (مثل مجموعة التعليمات - ISA) تُحدد كيفية تنفيذ الأوامر. لغة الآلة هي تمثيل مباشر لهذه البنية، لذا يجب أن تتوافق مع تصميم المعالج الداخلي (كعدد المسجلات، أنواع العمليات، وغيرها) حتى يفهمها ويُنفذها بشكل صحيح. هذا الاختلاف يمنع توافق البرامج بين معالجات مختلفة دون تعديل أو ترجمة.

High-Level Programming Languages

المستويات الأخرى من لغات البرمجة أنشأت لإنجاز الأهداف المختلفة

على سبيل المثال: جعلها أسهل على الإنسان ليقراً/ يكتب البرامج:

Other levels of programming languages were created to satisfy different objectives, e.g., make it easier for a human being to read/write programs:

- assembly languages (لغة المجمع)
- intermediate languages (اللغات الوسيطة)
- high-level languages (اللغات عالية المستوى)

Assembly Language

or simply assembly is a human-readable notation for the machine language.

It's much easier to remember: `movl %al, 97` than

10110000 01100001

```
movl (%edx,%eax),
%ecx
movl 12(%ebp), %eax
leal 0(%eax,4), %edx
movl $nodes, %eax
movl (%edx,%eax),
%eax
fldl (%ecx)
fsubl (%eax)
movl 8(%ebp), %eax
leal 0(%eax,4), %edx
movl $nodes, %eax
movl (%edx,%eax),
```

Example assembly code fragment

لغة المجمع أو التجميع البسيط هو تدوين يمكن قراءته من قبل الإنسان للغة الآلة



- لغة البرمجة عالية المستوى تمكن المبرمج من تحديد مستوى عالٍ (قريب من اللغة الطبيعية)، وما هي البيانات التي سيعمل عليها الكمبيوتر، وكيف سيتم تخزين هذه البيانات، وما هي الإجراءات التي يجب اتخاذها في ظل ظروف مختلفة.

(اللغة عالية المستوى مستقلة عن وحدة المعالجة المركزية) A high-level language is independent of CPU

```
int celsiusTemp = 32;
double fahrenheitTemp;

fahrenheitTemp = celsiusTemp *
9.0 / 5.0 + 32;

if (fahrenheitTemp > 100)
    System.out.println ("Hot !");
else
    System.out.println ("OK !");
```

Example Java Code fragment

Programming Languages

1. البرامج واللغة العالية المستوى:

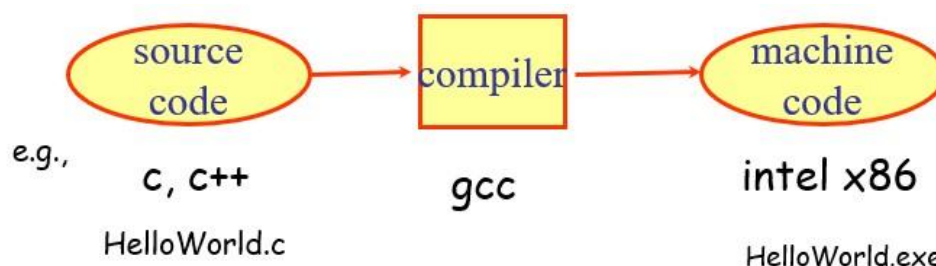
- البرنامج المكتوب بلغة عالية المستوى لا يمكن تشغيله مباشرة من قبل وحدة المعالجة (CPU).
- لذلك، يجب ترجمته إلى لغة الآلة (Machine Language).

2. ما هو الـ Compiler ؟

- الـ Compiler هو أداة برمجية.
- يقوم بترجمة الكود المصدري (Source Code) إلى لغة هدف (Target Language).

3. ما هي اللغة الهدف عادةً؟

- اللغة الهدف تكون غالباً لغة الآلة الخاصة بنوع معين من وحدات المعالجة (CPU).
- يعني كل مترجم ينتج كود يناسب نوع CPU محدد.



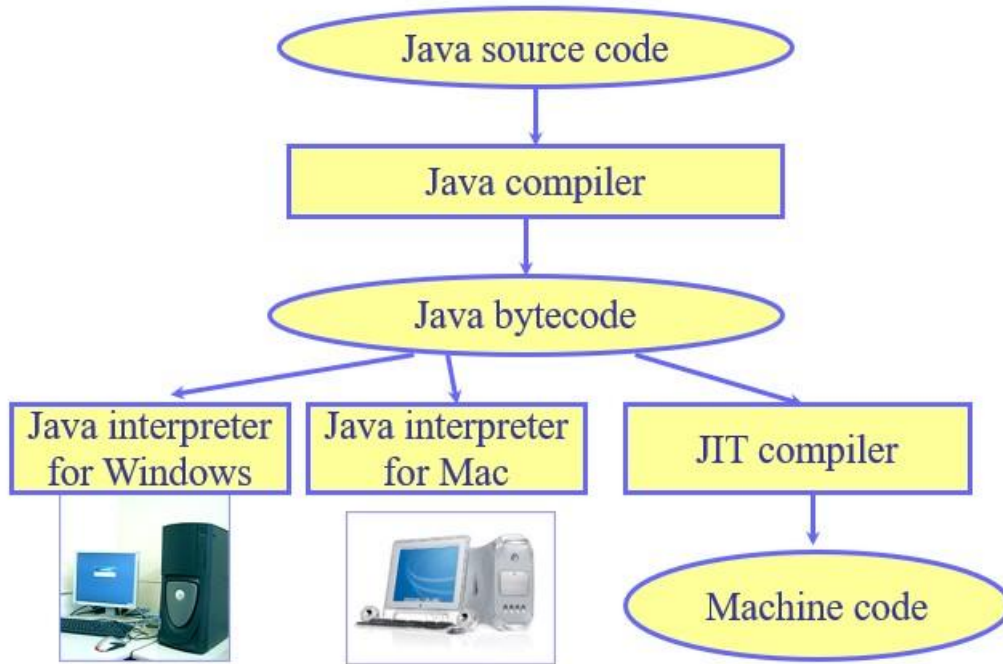
Java Translation

- لكي تكون Java **مستقلة عن المنصة** (platform independent) ، فهي لا تستطيع استخدام النهج التقليدي السابق الذي يعتمد على الترجمة المباشرة إلى machine language خاصة بوحدة معالجة مركزية معينة.
- لذلك، قدمت Java حلاً بديلاً يتمثل في إدخال لغة وسيطة (intermediate language) تُعرف باسم bytecode.
- هذه الـ bytecode ليست لغة آلة حقيقية لأي (traditional CPU)، بل يتم تنفيذها من خلال آلة افتراضية (virtual machine) تُعرف بـ JVM (Java Virtual Machine).
- يقوم Java compiler بترجمة الكود المصدري المكتوب في ملفات .java إلى bytecode ويتم حفظها ضمن ملفات .class.
- ونتيجة لذلك، فإن المترجم (compiler) لا يكون مرتبطاً بأي جهاز محدد أو معمارية معينة، لأنه لا ينتج machine code خاص بجهاز معين.
- وبالتالي، تعتبر Java **محايدة معمارياً** (architecture-neutral) ، إذ يمكن تنفيذ برامجها على أي نظام يحتوي على JVM، دون الحاجة إلى إعادة الترجمة.

Java Execution

- لتنفيذ برنامج مكتوب بلغة Java ، نحتاج إلى جزء برمجي إضافي يُسمى interpreter ، وهو المسؤول عن الترجمة من bytecode إلى machine language.
- هذا الـ interpreter يكون خاصاً بـ machine language معينة، أي أنه مصمم ليعمل على معالج (CPU) معين فقط.
- يعمل الـ interpreter على فهم bytecode الخاص بجافا، ثم يصدر التعليمات المناسبة بلغة الآلة التي يعرفها، ليقوم المعالج بتنفيذها.
- ونقول أيضاً إن الـ interpreter هو من يوفر البيئة التنفيذية المسماة بـ Java Virtual Machine (JVM) ، حيث يقوم بتمثيل هذه الآلة الافتراضية التي تشغل الكود الوسيط (bytecode).
- وهناك أسلوب آخر يُستخدم مع JVM ، وهو ما يُعرف بـ Just-In-Time compiler (JIT compiler) ، وهو يقوم بترجمة bytecode إلى machine code أثناء وقت التشغيل (on the fly) ، بالتوازي مع عملية التفسير، مما يؤدي إلى تحسين الأداء مقارنة بالتفسير فقط.

Java Translation and Execution



Related Topic: Programming Errors...

يمكن أن يحتوي البرنامج على ثلاثة أنواع رئيسية من الأخطاء (errors) تؤثر على تنفيذ وسلوك البرنامج.

1. النوع الأول هو أخطاء وقت الترجمة (compile-time errors) ، وهذه الأخطاء يكتشفها المترجم (compiler) عند تدقيق الكود المصدري، وتشمل مشاكل في syntax أو قواعد الكتابة وأخطاء أساسية أخرى.
■ في حالة وجود هذه الأخطاء، لا يتم إنشاء نسخة تنفيذية (executable) من البرنامج، مما يعني أنه لا يمكن تشغيل البرنامج قبل إصلاحها.
2. النوع الثاني هو أخطاء وقت التشغيل (run-time errors) ، التي تظهر أثناء تنفيذ البرنامج، مثل محاولة division by zero (القسمة على صفر)، والتي تؤدي إلى إنهاء البرنامج بطريقة غير طبيعية أو غير متوقعة.
3. النوع الثالث هو الأخطاء المنطقية (logical errors) ، حيث يعمل البرنامج وينفذ بدون توقف، لكنه ينتج نتائج غير صحيحة أو خاطئة بسبب خلل في منطق البرنامج.

Java Programming: The Edit/Compile/Run Loop

- تتكون عملية البرمجة في Java من ثلاث مهام رئيسية يجب تنفيذها بشكل متكرر ضمن دورة تطوير البرنامج.
- أولاً: تحرير الكود المصدري (edit Java source code) والذي يكون محفوظاً في ملفات بامتداد .java.
- ثانياً: تجميع الكود المصدري (compile Java source code) لإنتاج bytecode يُخزن في ملفات بامتداد .class.
- ثالثاً: تنفيذ وتشغيل واختبار (execute/run/test) الـ bytecode باستخدام interpreter الخاص بـ Java.

Comments

Two types of comments in Java single-line comments use //...

نوعان من التعليقات في جافا- تعليق على سطر واحد استخدم (اثنان سلاش) //

```
// this comment runs to the end of the line
```

multi-lines comments use /* ... */

للتعليق على عدة أسطر نستخدم /*.....*/ (سلاش نجمة، نجمة سلاش)

```
/* this comment runs to the terminating
symbol, even across line breaks */
```

Comments are ignored by the compiler:

- يتم تجاهل التعليقات من قبل الكومبايلر
- تستخدم فقط ليقراها البشر (أي الوثائق المضمنة)
- يجب تضمينها لشرح الغرض من البرنامج ووصف خطوات المعالجة

Identifiers

- المعرفات هي الكلمات التي يستخدمها المبرمج داخل البرنامج لتسمية المتغيرات، الدوال، أو الكائنات.
- يمكن أن يتكوّن المعرف من أحرف (letters)، أرقام (digits)، حرف الشريط السفلي (underscore) (_) ، وعلامة الدولار (\$).
- لا يمكن للمعرف أن يبدأ برقم، وذلك لأن اللغة تفرق بين المعرفات والأرقام لتجنب اللبس في التحليل البرمجي.
- لغة Java حساسة لحالة الأحرف (case sensitive)، لذا يعتبر المعرفان Total و total معرفين مختلفين تماماً.

Identifiers: Reserved Words

- توجد معرفات خاصة تعرف بـ reserved words أو الكلمات المحجوزة، وهي كلمات لها معنى محدد مسبقاً ومُعرف في لغة Java.
- لا يمكن استخدام هذه الكلمات المحجوزة لأي غرض آخر داخل البرنامج، فهي محجوزة لأغراض اللغة نفسها ولا يمكن إعادة تعريفها أو استخدامها كمعرفات عادية.

abstract	default	goto	operator	synchronized
boolean	do	if	outer	this
break	double	implements	package	throw
byte	else	import	private	throws
byvalue	extends	inner	protected	transient
case	false	instanceof	public	true
cast	final	int	rest	try
catch	finally	interface	return	var
char	float	long	short	void
class	for	native	static	volatile
const	future	new	super	while
continue	generic	null	switch	

Java Program Structure

- في لغة Java، يتكون البرنامج من صف (class) واحد أو أكثر.
- يحتوي كل class على تابع (method) واحد أو أكثر.
- يجب أن يحتوي تطبيق Java دائماً على تابع خاص يسمى main، وهو نقطة البداية لتنفيذ البرنامج.
- يحتوي التابع على تعليمات البرنامج (program statements) التي تحدد سلوك البرنامج.

Java Program Structure: Class

```
// comments about the class
public class HelloWorld
{
}
```

class name is an identifier; convention we follow: capitalize each English word

class header

class body starts with a left brace { and ends with a right brace }

comments can be added almost anywhere

Structure of a Java Class

```
public class Classroom
{
    // instance variables
    int min_capacity = 0;
    int max_capacity = 100;
    int chairs = 30;
    ...
    // methods
    public int addStudents (int students)
    { ... }
    public int addChairs (int chairs)
    { ... }
    ...
    public static void main (String[] args)
    { ... }
}
```

Java Method and Statements

التوابع (Methods) ✓

- التوابع هي وحدات بناء الصف (building blocks of a class).
- كل اسم تابع هو معرف (identifier) ، مثل addStudents :
- الاصطلاح المتبع (convention) هو أن يبدأ اسم التابع بحرف صغير، وكل كلمة إنجليزية إضافية تبدأ بحرف كبير، مثل main ، doMyJob.
- التابع الأساسي في أي برنامج جافا هو التابع main :
- يجب أن يحتوي كل تطبيق جافا على تابع main واحد فقط.
- تبدأ جميع البرامج بتنفيذ التابع main كالنقطة الأولى لبدء عمل البرنامج.
- تستخدم الأقواس المعقوفة { } لبدء وإنهاء التابع.

التعليمات (Statements) ✓

- يجب أن تنتهي كل تعليمة (statement) بفاصلة منقوطة ;

Classes and Methods

- تعد الصفوف (classes) والتوابع (methods) من البنى اللغوية (language constructs) في لغة Java ، وهي مصممة لمساعدة المبرمج على تنظيم البرنامج بشكل أفضل.
- يقوم التابع (method) بتنظيم سلسلة من التعليمات (statements) ، بحيث يمكننا استخدام تابع واحد للإشارة إلى مجموعة متكاملة من العمليات.
- بالمثل، يقوم الصف (class) بتنظيم مجموعة من التوابع (methods) والمعلومات (data) ذات الصلة، مما يساعد في تجميع الوظائف المرتبطة معًا.
- إن تقسيم المهام إلى صفوف وتوابع يُعتبر من أساسيات البرمجة كائنية التوجه (object-oriented programming)، حيث يساهم ذلك في إنشاء حدود تجريدية (abstraction boundaries) واضحة بين المهام المختلفة والكائنات داخل البرنامج.

Classes and Methods Provide Abstraction

■ مفهوم التجريد: (Abstraction)

- في تصميم البرمجيات (program design) ، تُعتبر الصفوف (classes) والتوابع (methods) هياكل أساسية تُستخدم بهدف توفير التجريد. (abstraction)
- التجريد يعني إخفاء التفاصيل غير الضرورية والتركيز فقط على ما هو مهم، بحيث يتمكن المبرمج من التعامل مع المهام بشكل أبسط وأكثر تركيزًا.

■ ما الهدف من التجريد؟

- الهدف الأساسي هو مساعدة المبرمجين على تجاهل التفاصيل المعقدة مؤقتًا عند الحاجة، والتركيز على الصورة العامة أو المهمة الحالية.
- البشر يمكنهم التعامل مع 7 وحدات * 2 من المعلومات فقط في نفس الوقت (حسب علم النفس الإدراكي).

■ كيف نحقق التجريد؟

- من خلال تنظيم الكود إلى صفوف وتوابع، بحيث:
- يقوم كل method بتنفيذ مهمة محددة.
- وكل class يحتوي على مجموعة من التوابع المرتبطة مع البيانات الخاصة بها.
- هذا التقسيم المنظم (modularization) يساعدنا على بناء برامج أكبر وأكثر تعقيدًا دون الوقوع في فوضى التفاصيل.

Methods and Classes

التوابع (Methods)

- التابع (method) هو وسيلة لتوفير تجريد (abstraction) لسلسلة من التعليمات (statements).
- في البداية، يُستخدم التابع لمساعدة المبرمج على الإشارة إلى سلسلة تعليمات قابلة لإعادة الاستخدام، مما يقلل من التكرار ويزيد التنظيم.
- بشكل أوسع، يمثل التابع خدمة (service)، إذ يقوم بعمل محدد عندما يُطلب منه ذلك.

مكونات التابع:

- يأخذ مدخلات (parameters).
- ينفذ إجراءات (actions).
- ويقوم أحياناً بإرجاع قيمة (return value).

طريقة الاستخدام:

- نستدعي التابع (invoke/call) عندما نرغب بتنفيذ خدمته.
- يتم تمرير المعاملات (parameters) المناسبة أثناء الاستدعاء.

الصفوف (Classes)

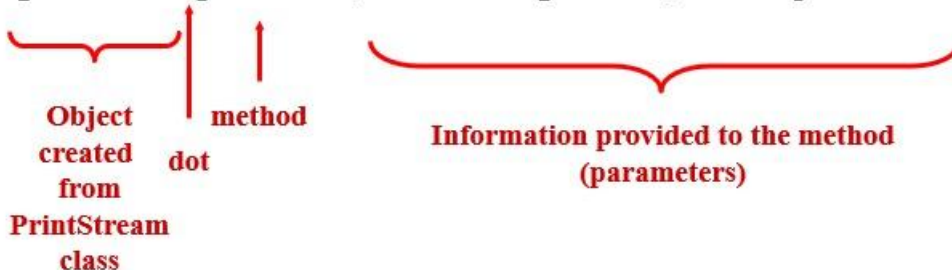
- الصف (class) يوفر تجريداً للأشياء (objects) التي تتشارك سلوكيات (behaviors) أو خدمات (services) معينة.
- نستطيع أن نطلب من الكائنات (objects) القيام بخدمات معينة، وهذه الخدمات تُعرّف ضمن التوابع الموجودة في الصف.

العلاقة بين الصف والكائن:

- الصف يُعتبر بمثابة مخطط (blueprint) للكائنات.
- نُنشئ الكائنات (instantiate objects) من خلال تعريف الصف.

Example

```
System.out.println ("Whatever you are, be a good one.");
```



Declaring Object Reference Variables

■ إنشاء الكائنات: (Creating Objects)

- في الحالة العامة، نحتاج أولاً إلى إنشاء الكائنات (objects) يدوياً باستخدام الكود.
- توجد كائنات منشأة مسبقاً (pre-created) مثل System.out، لكنها حالات نادرة الاستخدام.

■ تتبع الكائنات باستخدام المتغيرات: (Tracking Objects with Variables)

- حتى تتمكن من تتبع الكائنات التي أنشأناها، نحتاج إلى متغيرات (variables) نقوم بالإشارة إليها (refer to them).
- هذه المتغيرات هي عبارة عن معرفات (identifiers) يمكن من خلالها الوصول إلى الكائنات في الذاكرة.
- من المهم أن نعلم أن المتغير الواحد يمكنه أن يشير إلى كائنات مختلفة (واحد تلو الآخر) خلال فترة تنفيذ البرنامج.

■ تحديد نوع المتغير: (Variable Type)

- لتجنب حدوث خلط بين أنواع الكائنات المختلفة (mixing object types)، من الضروري تحديد نوع الكائن (Type) الذي يمكن للمتغير الإشارة إليه.
- ويتم ذلك باستخدام الصيغة التالية: `<variableName> <Type>;`

Example

■ عند كتابة السطر:

`String title;`

فهنا `String` هي class معرفة مسبقاً في Java، و `title` يُعتبر object reference variable أي متغير مرجعي يشير إلى كائن.

- هذا السطر لا يقوم بإنشاء object فعلي، وإنما فقط يعلن عن متغير يمكنه لاحقاً الإشارة إلى كائن.
- لإنشاء الـ object نفسه، يجب استخدام خطوة إضافية يتم فيها التهيئة بشكل صريح.

Explicitly Creating Objects

- We use the new operator to create an object

```
title = new String ("Java Software Solutions");
```

This calls the **constructor** method of class **String**, which is a special method that sets up the object (every class must have a constructor method)

Now the variable title refers to an instance of String object

title → "Java Software Solutions"

```
title = new String ("Java Software Solutions");
```

- “new” operator in general:

```
<varName> = new <className> ([<params>]);
```

This calls the **constructor** method of class **<className>** which (optionally) takes some parameters **<params>**

Now the variable <varName> refers to an object which is an instance of class <className>

<varName> → Object of class <className>, constructed using parameters <params>

Use Methods of Objects

```
title = new String ("Java Software Solutions");
```

title → "Java Software Solutions"

- Once an object has been instantiated, we can use the dot operator to invoke its methods, e.g.,

```
count = title.length();
```

بمجرد إنشاء مثيل للكائن، يمكننا استخدام عامل النقطة لاستدعاء التابع

```
count = title.length();
```

- This **procedure call** invokes **method** “length” of class String, because “title” is a **variable** which refers to an **object** which is an **instance of** class String.

- استدعاء الإجراء يستدعي التابع "length" من الصف string لأن "title" متغير يشير إلى كائن هو مثيل من صف string

- Saying it another way: “title” is a variable of “type”String

- بعبارة أخرى: "title" متغير من النوع string

- Methods (e.g. method “length” of class String) can return values (Here the return value is assigned to variable “count”)

انتهت المحاضرة

